ARIA plant E3 project

End-to-end workflow and package operator guide

Release	0.8.0
Date	24 July 2026
Project	PT_E3_6 / ARIA plant E3
Primary entry point	E3_project_draft/run_e3_pipeline.sh

This guide covers Slurm and non-Slurm operation, configuration, directory organisation, adding a new dataset, restart controls, monitoring, outputs and a quick start for every package.

Contents

This repository contains the modular discovery, orthology, expression, structural analysis, integration and reporting software for the PT_E3_6 / ARIA plant E3 project. The packages remain independently testable, but `e3_end_to_end_workflow` joins them into one restartable Snakemake workflow.

The main user entry point is at the top of the repository:

```
E3_project_draft/run_e3_pipeline.sh
```

It can run the complete configured pipeline in either of two recommended ways:

1. `--mode slurm`: submit the Snakemake controller itself as a Slurm batch job, then leave it

running after logout; or

1. `--mode local`: run the complete enabled DAG in the foreground on a machine without Slurm.

The repository never treats the presence of an output file as proof that a stage completed. Checksum-bound manifests, controlled configuration, declared output validation and atomic publication are the restart authority.

Sixty-second cluster quick start

From the repository root:

```
cd /home/pthorpe001/data/2026_E3_protac/E3_project_draft
conda activate e3_end_to_end_workflow

./run_e3_pipeline.sh \
    --mode slurm \
    --config e3_end_to_end_workflow/config/my_immutable_run.yaml \
    --max-jobs 4 \
    --account barton \
    --partition general \
    --resume
```

The submission command returns after Slurm accepts the controller. You can close the terminal or lose the network connection. Both the Snakemake controller and its scientific stage jobs are then owned by Slurm.

Check it later:

```
./run_e3_pipeline.sh \
    --mode slurm \
    --config e3_end_to_end_workflow/config/my_immutable_run.yaml \
    --status

squeue -u "${USER}"
```

The submission response prints the persistent controller log. Follow it with:

```
tail -F /absolute/path/to/controller_slurm_JOB_ID.log
```

Do not use `--stop-after` when the intention is to run the whole configured pipeline. Every stage whose YAML entry has `enabled: true` will run when its prerequisites are ready.

Sixty-second local quick start

Use this mode on a workstation or another system without Slurm:

```
cd /path/to/E3_project_draft
conda activate e3_end_to_end_workflow

./run_e3_pipeline.sh \
    --mode local \
    --config e3_end_to_end_workflow/config/my_immutable_run.yaml \
    --threads 8 \
    --resume
```

Local mode remains in the foreground. Closing the terminal stops the controller. For a quick installation check, use the synthetic configuration:

```
./run_e3_pipeline.sh \
    --mode local \
    --config e3_end_to_end_workflow/config/synthetic.yaml \
    --threads 4 \
    --dry-run
```

Synthetic outputs are marked as test data and are not production eligible.

Execution modes

Root mode	Controller location	Scientific jobs	Survives logout	Intended use
`slurm`	Small Slurm batch job	Separate Slurm jobs	Yes	Recommended Dundee cluster mode
`local`	Current terminal	Local processes	No	Workstations, development and synthetic tests
`login-detached`	Detached login-node process	Separate Slurm jobs	Usually	Legacy fallback only where login-node policy permits

The default is `--mode slurm`. The controller allocation defaults to one CPU, 4,000 MiB and three days. Override these separately from scientific resources when required:

```
./run_e3_pipeline.sh \
    --mode slurm \
    --config e3_end_to_end_workflow/config/my_immutable_run.yaml \
    --controller-memory-mb 6000 \
    --controller-runtime 2-00:00:00 \
    --max-jobs 4 \
    --account barton \
    --partition general \
    --resume
```

`--controller-runtime` must cover the orchestration period, including time spent waiting for child jobs. The Dundee maximum is three days. If a much larger analysis cannot finish within that controller window, resume the same immutable run after the controller ends; validated completed stages are retained.

First installation

Install the master environment and package:

```
cd /home/pthorpe001/data/2026_E3_protac/E3_project_draft/e3_end_to_end_workflow

conda env create --file environment.yml
conda run --name e3_end_to_end_workflow \
    python -m pip install --no-deps --editable .
conda run --name e3_end_to_end_workflow ./run_tests.sh
```

The environment supplies Python, Snakemake 9, the Slurm executor plugin, DuckDB, MAFFT and the project-approved OrthoFinder 2.5.5. Component packages invoked through their own Conda environment, such as `e3_orthology` and `e3_structural_alignment`, must also be installed when the selected YAML uses those adapters.

Validate the repository-level launcher:

```
cd ..
./run_repository_tests.sh
```

Which configuration template to use

Do not edit a completed run's configuration into a different analysis. Copy a template to a new, descriptive, immutable filename.

Analysis	Starting template
Current reviewed analysis using the authoritative 60-proteome February OrthoFinder result and existing evidence	`e3_end_to_end_workflow/config/grant_aligned_reuse.cluster.template.yaml`
New or expanded proteome panel requiring fresh discovery and OrthoFinder	`e3_end_to_end_workflow/config/production.cluster.template.yaml`
Small fresh OrthoFinder validation only	`e3_end_to_end_workflow/config/five_proteome_orthofinder.cluster.yaml`
Software test	`e3_end_to_end_workflow/config/synthetic.yaml`

Example:

```
cd e3_end_to_end_workflow

cp \
    config/production.cluster.template.yaml \
    config/my_species_panel_v0_1_0_20260724.yaml
```

Replace every applicable `CHANGE_ME` value. A placeholder must never be accepted as a production default.

Required end-to-end configuration

The YAML has six main sections.

Section	Required content
`schema_version`	Configuration schema version; currently `1`
`run`	Unique run name, `production` or `synthetic` mode, repository root and output root
`inputs`	Controlled manifests and reusable evidence authorities required by enabled stages
`analysis`	Scientific thresholds, species priorities and feature-specific policies
`benchmarking` / `reporting`	Resource sampling and bounded HTML preview settings
`stages`	Enabled state, evidence mode, command adapter, expected outputs and Slurm resources

`run`

Use a run name that encodes the dataset or purpose, method version and date:

```
run:
  name: wheat_barley_rice_panel_v0_1_0_20260724
  mode: production
  project_root: /home/pthorpe001/data/2026_E3_protac/E3_project_draft
  output_root: /home/pthorpe001/data/2026_E3_protac/analysis/e3_end_to_end_runs
```

A biologically different panel needs a different `run.name`. Never reuse a formal run name merely to avoid creating another output directory.

`inputs`

The exact files needed depend on enabled branches and their evidence modes.

Input	Purpose
`proteomes_manifest`	Species-to-FASTA mapping for fresh proteome preparation, discovery and OrthoFinder
`seeds_manifest`	Controlled known-E3 seed evidence
`shortlist_manifest`	Optional reviewed shortlist; not used by the computational gate unless configured
`candidate_evidence` / manifest	Reused candidate-cluster evidence for the reviewed-results route
`orthofinder_archive`	Read-only reviewed OrthoFinder archive for reuse

Input	Purpose
`orthology_species_manifest`	Maps OrthoFinder species columns to project species identifiers
`inherited_sqlite`	Read-only regression authority where the configured orthology adapter uses it
`expression_manifest`	Checksummed existing Expression Atlas Parquet resources
`ligandability_manifest`	Checksummed existing pocket/model resources for reuse
`domain_annotation_manifest`	Optional pre-existing domain resource manifest
`domain_cache_root`	Shared incremental InterPro/Pfam response cache
`e3_domain_catalogue`	Reviewed catalogue used to interpret E3-associated domain evidence

Disabled branches do not require irrelevant placeholder inputs. Enabled production stages fail closed when a required authority is absent, empty, malformed or checksum-inconsistent.

Proteome manifest

Start with `config/proteomes.template.tsv`. It is tab-separated:

```
species_id scientific_name fasta_path fasta_sha256 include
arabidopsis_thaliana Arabidopsis thaliana /data/proteomes/Arabidopsis_thaliana.faa <sha256> true
```

Rules:

1. one row per complete proteome;
1. stable unique `species_id` values;
1. absolute FASTA paths for cluster runs;
1. SHA-256 calculated from the exact file being analysed;
1. `include` explicitly `true` or `false`;
1. no comma-separated analytical manifest;
1. do not mix protein releases silently under one run name.

Calculate a checksum with:

```
sha256sum /data/proteomes/Arabidopsis_thaliana.faa
```

Known-E3 seed manifest

The packaged controlled seed evidence is normally used:

```
seeds_manifest: ../data/known_e3_seed_evidence.tsv.gz
```

For a revised seed authority, use the schema in `config/known_e3_seed_evidence.template.tsv`, preserve the source path and source row, and create a new versioned resource. Changing the seeds changes the discovery question and therefore requires a new run name.

Orthology species manifest

This file connects OrthoFinder's run-specific species representation to the project names used by domain, expression, prioritisation and reporting stages. It must describe every included proteome. An expanded panel needs a new species manifest and a new OrthoFinder result. Do not carry `OG...` or hierarchical-group labels from an earlier species panel into a new run merely because the labels look similar.

Existing expression data

The current reviewed run reuses:

```
analysis/expression_atlas_ftp_full/
    manifests/e3_workflow_expression_resources.tsv
    parquet/atlas_expression_long/
    parquet/atlas_sample_metadata_long/
    parquet/atlas_sample_metadata_wide/
    e3_expression.duckdb
```

Stage `07_expression` does not redownload these resources when its `evidence_mode` is `reuse`. It performs new run-specific identifier mapping and expression summaries for the selected OrthoFinder group members.

`analysis`

Review, rather than blindly copy:

1. `analysis.prioritisation.target_species`;
1. `analysis.prioritisation.mandatory_species`;
1. evidence completeness thresholds;
1. ranking weights and shortlist sizes;
1. domain download/cache settings;
1. ligandability mapping and conservation thresholds; and
1. optional structural-alignment thresholds.

The target and mandatory species names must agree with the controlled species manifests. Keep `analysis.structural_alignment.use_for_prioritisation: false` until the selected 3D thresholds have been reviewed on an appropriate multi-structure dataset.

`stages`

Each stage has:

```
enabled: true
required: true
evidence_mode: reuse
expected_outputs:
  - tables/example.parquet
threads: 8
memory_mb: 64000
runtime_minutes: 1440
```

Fresh external components also have a YAML argument-vector `command`. Commands are lists, not shell strings, so paths and options are passed without shell interpolation. The permitted evidence modes are stage-specific and include `validate`, `prepare`, `reuse`, `download`, `derive`, `generate` and `disabled`.

On the Dundee cluster, no stage should request more than 4,320 minutes. If a component genuinely needs longer, split it into restartable component work rather than requesting an impossible walltime.

Recommended organisation for a new dataset

Keep code, controlled inputs, reusable evidence and run outputs separate:

```
/home/pthorpe001/data/2026_E3_protac/
    E3_project_draft/                 # Git checkout and software only
        README.md
        run_e3_pipeline.sh
        e3_end_to_end_workflow/
        component packages...
    inputs/
        wheat_barley_rice_v1/
            manifests/
                proteomes.tsv
                orthology_species.tsv
            proteomes/
                Triticum_aestivum.faa
                Hordeum_vulgare.faa
                Oryza_sativa.faa
    analysis/
        e3_resource_cache/interpro/
        expression_atlas_ftp_full/
        e3_end_to_end_runs/
            wheat_barley_rice_panel_v0_1_0_20260724/
    retained_authorities/
        read_only_source_material...
```

The checkout should not accumulate scientific outputs. The end-to-end `run.output_root` is where formal runs, logs, staging data, failed attempts and reports belong. Existing inherited evidence remains read-only.

Adding a new or different dataset

Use this sequence.

1. Decide whether the analysis is a reviewed reuse or a genuinely fresh panel.
2. Create a versioned input directory outside the Git checkout.
3. Obtain one complete protein FASTA per species and record its release/source.
4. Calculate SHA-256 checksums and build the proteome manifest.
5. Build a matching orthology species manifest.
6. Decide whether the existing controlled seed evidence still answers the intended question.
7. Confirm expression coverage. Add species to `expression_downloader/data/species.txt` and build missing resources only when required.

1. Copy `config/production.cluster.template.yaml` to a new run-specific YAML.
2. Set a unique `run.name`, input paths, target species, mandatory species and stage resources.
3. Replace fresh-stage adapter placeholders with reviewed named-option command vectors.
4. Keep OrthoFinder at the project-approved 2.5.5 for comparability unless a new method decision is explicitly documented.

1. Validate the configuration and human-readable plan.
2. Perform a dry run.
3. Run a small representative panel before the full dataset.
4. Submit the full run through the repository-root Slurm mode.

Validation commands:

```
cd /home/pthorpe001/data/2026_E3_protac/E3_project_draft
conda activate e3_end_to_end_workflow

e3-workflow validate \
    --config e3_end_to_end_workflow/config/my_new_panel.yaml

e3-workflow plan \
    --config e3_end_to_end_workflow/config/my_new_panel.yaml \
    --human

./run_e3_pipeline.sh \
    --mode local \
    --config e3_end_to_end_workflow/config/my_new_panel.yaml \
    --dry-run
```

The local dry run creates no scientific jobs. A cluster-specific dry run can instead be submitted through `--mode slurm --dry-run`.

Whole-pipeline stages

Stage	Primary owner	Purpose
`00_inputs`	Master workflow	Validate and checksum controlled inputs
`01_prepared_proteomes`	Master adapter	Validate and isolate included protein FASTAs
`02_discovery`	`e3_discovery_engine`	E3-seeded DIAMOND/DeepClust discovery or reviewed reuse
`03_candidate_evidence`	`e3_source_to_parquet_seed`	Build/reuse candidate cluster evidence
`04_orthofinder`	OrthoFinder 2.5.5	Generate or validate complete-proteome group results
`05_orthology`	`e3_orthology_integration`	Reconcile identifiers and publish candidate-relevant group membership/sequences
`06_domains`	Master domain adapter	Retrieve/reuse InterPro/Pfam annotations and explicit coverage states
`07_expression`	Master expression adapter	Map group members to existing Expression Atlas resources
`08_shortlist_gate`	Master prioritisation	Build the transparent computational structural shortlist
`09_ligandability`	`e3_ligandability_pipeline` plus master adapter	Select pockets and assess pocket-region conservation
`09b_structural_alignment`	`e3_structural_alignment`	Optional US-align/TM-align 3D pocket-position evidence
`10_integrated_resource`	Master workflow	Create integrated DuckDB, Parquet, TSV and scientific report
`11_app_ready`	Master workflow	Publish Python/Shiny hand-off configuration

Independent branches run concurrently when resources permit. For example, domain and expression work can run in parallel after orthology publication. DeepClust sequence clusters and OrthoFinder groups remain different scientific concepts.

Resume and controlled reruns

Normal restart:

```
./run_e3_pipeline.sh \
    --mode slurm \
    --config e3_end_to_end_workflow/config/my_immutable_run.yaml \
    --resume
```

Bound a diagnostic run:

```
./run_e3_pipeline.sh \
    --mode slurm \
    --config e3_end_to_end_workflow/config/my_immutable_run.yaml \
    --resume \
    --stop-after 05_orthology
```

Intentionally rerun from a stage:

```
./run_e3_pipeline.sh \
    --mode slurm \
    --config e3_end_to_end_workflow/config/my_immutable_run.yaml \
    --start-at 07_expression \
    --resume
```

Force one stage:

```
./run_e3_pipeline.sh \
    --mode slurm \
    --config e3_end_to_end_workflow/config/my_immutable_run.yaml \
    --force-stage 07_expression \
    --resume
```

`--resume` never means "skip because a filename exists". A completed stage is reusable only when its manifest, configuration/input digests and declared output checksums validate. Failed staging directories are retained under `failed/` for diagnosis and do not satisfy the stage contract. Intentional reruns preserve invalidated or superseded results rather than silently overwriting them.

Monitoring and logs

Controller state:

```
./run_e3_pipeline.sh \
    --mode slurm \
    --config e3_end_to_end_workflow/config/my_immutable_run.yaml \
    --status
```

All jobs:

```
squeue -u "${USER}"
```

Completed controller or stage accounting:

```
sacct \
    --jobs JOB_ID \
    --format JobID,JobName,State,ExitCode,Elapsed,MaxRSS,AllocCPUS
```

Important persistent paths below the configured run:

```
workflow_control/controller.slurm.tsv
workflow_control/controller.lock
workflow_logs/controller_slurm_JOB_ID.log
workflow_logs/STAGE.snakemake.log
STAGE/stage_manifest.json
STAGE/report/stage_report.html
benchmark_summary/
reports/e3_workflow_summary.html
```

The `.snakemake/slurm_logs` area is executor-managed and may be transient. Prefer the persistent `workflow_logs` and stage manifests for diagnosis.

Final outputs

The two main final analytical authorities are:

```
10_integrated_resource/duckdb/e3_integrated_resource.duckdb
10_integrated_resource/tables/e3_candidate_master_results.parquet
```

The DuckDB retains one-to-many group members, sequences, domains, expression mappings, pockets and residue-level evidence. The master Parquet provides one wide row per candidate group for portable review. TSV is used for human-auditable tables; comma-separated analytical outputs are not used.

The complete run also publishes:

1. stage-level HTML reports;
1. one consolidated workflow HTML report;
1. benchmark and Slurm accounting summaries;
1. candidate rankings and missing-evidence states;
1. application-ready configuration; and
1. checksummed provenance/manifests.

Package quick starts

The end-to-end runner is the normal route. Run a component directly only for development, component-specific validation or resource preparation.

`e3_end_to_end_workflow`

Purpose: stable DAG, restart state, stage adapters, integration, benchmarking and reports.

```
cd e3_end_to_end_workflow
conda env create --file environment.yml
conda run --name e3_end_to_end_workflow \
    python -m pip install --no-deps --editable .
conda run --name e3_end_to_end_workflow ./run_tests.sh
```

Then return to the repository root and use `./run_e3_pipeline.sh`.

`e3_discovery_engine`

Purpose: E3-seeded DIAMOND/DeepClust discovery and candidate-cluster resource.

```
cd e3_discovery_engine
conda env create --file workflow/envs/production.yml
conda run --name e3_discovery python -m pip install --editable .
conda run --name e3_discovery ./run_tests.sh

cp config/config.example.production.yaml config/config.production.yaml
cp config/samples.production.example.tsv config/samples.production.tsv
./run_workflow.sh config/config.production.yaml 16
```

Always dry-run a new configuration. The legacy DIAMOND environment is only for controlled reproduction and must not be mixed with production outputs.

`e3_source_to_parquet_seed`

Purpose: source-preserving Parquet/DuckDB conversion and candidate-evidence publication.

For the current candidate-evidence layer:

```
cd e3_source_to_parquet_seed
./run_tests.sh

./run_e3_candidate_evidence.sh \
    /path/to/e3_discovery_resource.duckdb \
    /path/to/derived_v0_4_0 \
    --conda-env e3_discovery
```

Older `run_e3_seed_pipeline.sh` examples use positional arguments and are retained for package compatibility. New master-workflow adapters should use named options.

e3_orthology_integration

Purpose: reconcile candidate identifiers with OrthoFinder groups and publish candidate-relevant group members and sequences.

```
cd e3_orthology_integration
conda env create --file environment.yml
conda run --name e3_orthology python -m pip install --no-deps --editable .
conda run --name e3_orthology ./run_tests.sh

./run_e3_orthology_integration.sh \
    --conda-env e3_orthology \
    --threads 4 \
    --dry-run
```

The package also has a standalone Slurm submitter. For the whole analysis, let the master workflow own stage submission.

expression_downloader

Purpose: discover/download Expression Atlas experiments and create reusable partitioned Parquet and DuckDB views.

```
cd expression_downloader
mamba env create --file envs/e3_atlas_duckplyr.yml
conda activate e3_atlas_duckplyr

R CMD INSTALL .
Rscript inst/scripts/08_run_tests.R
./inst/scripts/09_run_python_tests.sh

./inst/scripts/run_python_first_then_r.sh \
    --species_file=data/species.txt \
    --override_tsv=data/species_overrides.tsv \
    --output_dir=/path/to/analysis/expression_atlas \
    --force_download=false \
    --force_import=false \
    --create_duckdb=true \
    --import_backend=python \
    --expression_file_types=tpms,fpkms
```

Add one species per line to `data/species.txt`. Existing non-empty downloads are skipped unless forced. The end-to-end expression stage consumes a validated resource manifest rather than redownloading the data.

e3_ligandability_pipeline

Purpose: AlphaFold model confidence, FPocket/P2Rank pockets and residue mapping.

```
cd e3_ligandability_pipeline
conda env create --file environment.cluster.yml
conda run --name e3_ligandability python -m pip install --editable .
conda run --name e3_ligandability ./run_tests.sh
conda run --name e3_ligandability ./run_coverage.sh
```

The standalone runner currently retains a positional compatibility interface:

```
./run_e3_ligandability.sh \
    examples/accessions.local_models.example.tsv \
    /path/to/analysis/ligandability_smoke \
    config/config.cluster.yaml \
    e3_ligandability
```

For production end-to-end work, use the master YAML adapter and its declared output contract.

e3_structural_alignment

Purpose: optional direct US-align/TM-align comparison of pocket position and local residue conservation.

```
cd e3_structural_alignment
conda env create --file environment.yml
conda run --name e3_structural_alignment \
    python -m pip install --no-deps --editable .
conda run --name e3_structural_alignment ./run_tests.sh

./run_e3_structural_alignment.sh --help
```

The standalone interface uses named options. In normal use it is stage `09b` and may remain disabled when compatible structures are insufficient.

e3_python_app

Purpose: read-only Streamlit exploration of a completed result.

```
cd e3_python_app
python -m pip install --editable '.[dev]'
./run_tests.sh

./run_e3_python_app.sh \
    --resource-duckdb /path/to/e3_integrated_resource.duckdb \
    --max-rows 1000 \
    --host 127.0.0.1 \
    --port 8501
```

It can alternatively use `--resource-parquet` or `--resource-run-dir`.

E3_shiny_app

Purpose: read-only R Shiny exploration using the same integrated resource contract.

```
cd E3_shiny_app
Rscript inst/scripts/check_dependencies.R
Rscript inst/scripts/run_tests.R

./run_app.sh \
    --resource_duckdb_path /path/to/e3_integrated_resource.duckdb \
    --expression_duckdb_path /path/to/e3_expression.duckdb \
    --max_table_rows 1000 \
    --host 127.0.0.1 \
    --port 3838
```

It can alternatively use `--resource_parquet_path` or `--resource_run_dir`.

Package ownership summary

Package	Owns	Does not claim
`e3_discovery_engine`	Sequence clusters containing controlled E3 seeds	That every cluster member is an E3 ligase
`e3_source_to_parquet_seed`	Curated/source-preserving resources and candidate evidence	Orthology
`e3_orthology_integration`	Run-specific OrthoFinder membership and identifier reconciliation	Experimental orthology validation
`expression_downloader`	Expression Atlas acquisition/import	Protein activity
`e3_ligandability_pipeline`	Predicted structural confidence and cavities	Binding or degradative activity
`e3_structural_alignment`	Predicted 3D position/conservation comparisons	Ligand selectivity or biochemical function
`e3_end_to_end_workflow`	Orchestration, integration, provenance and transparent prioritisation	Experimental proof
Reporting apps	Read-only exploration	New scientific transformations

Tests

Repository-root launcher:

```
./run_repository_tests.sh
```

Master package:

```
conda run --name e3_end_to_end_workflow \
    ./e3_end_to_end_workflow/run_tests.sh
```

Each component has its own `run_tests.sh` or documented R test command. Run a component's tests after changing that component, and run the end-to-end synthetic regression after changing a publication contract used downstream.

Common failures

Controller is `PENDING`

Inspect:

```
squeue --jobs JOB_ID --start
scontrol show job JOB_ID
```

No second controller should be submitted for the same run while the first is pending.

Controller ended but some stages completed

Check the controller log and accounting, fix the cause, then submit the same immutable YAML with `--resume`. Do not delete validated successful stage directories.

Configuration path or input changed

Do not weaken validation. Either restore the immutable input or create a new run YAML and run name for the changed dataset.

A stage output exists but is rejected

Inspect its `stage_manifest.json`, persistent stage log and the retained `failed/` directory. Existence alone is intentionally insufficient.

Expression coverage is absent for a species

This should become an explicit unavailable or mapping state, not a false biological zero. Use `expression_downloader` to prepare missing reusable data if it exists in Expression Atlas.

Optional structural alignment is unavailable

Leave `09b_structural_alignment` disabled/optional. The final integration will record `NOT_ASSESSED`; it must not reinterpret missing structures as dissimilar pockets.

Further documentation

1. `docs/E3_PROJECT_OPERATOR_GUIDE_v0_8_0.pdf`: printable cross-package operator guide.
1. `README.md`: accessible source and continuously maintained quick-start guide.
1. `REPOSITORY_FILE_GUIDE.md`: file-by-file package map.
1. `e3_end_to_end_workflow/README.md`: detailed master package behaviour.
1. `e3_end_to_end_workflow/docs/ARCHITECTURE.md`: DAG and atomic-publication design.
1. `e3_end_to_end_workflow/docs/EVIDENCE_MODES_AND_SCALING.md`: reuse/fresh evidence policy.
1. each component package's `README.md`: complete standalone interface and scientific boundary.

Document control

Source: repository-root README.md in release 0.8.0. Commands must still be checked against the immutable run configuration and the current cluster policy before execution.